

PROSIT 1

Chapter 1 — Introduction to Distributed Architectures

JobBoard: a monolith facing the multi-device challenge

Objectives

By the end of this prosit, students will be able to:

- 01** Discover the functional context of the JobBoard application.
- 02** Analyse an existing application built on a monolithic architecture.
- 03** Identify the main features of the application.
- 04** Recognise the characteristics of tightly coupled (“spaghetti”) code.
- 05** Identify the limits of a monolithic architecture as requirements evolve.
- 06** Analyse the maintenance, scalability and deployment difficulties of a monolithic application.
- 07** Identify the actors and the main business processes of the application.
- 08** Propose architectural improvements, justifying the need to evolve towards a distributed architecture.

1 Context

TalentConnect is a young Tunisian company that runs **JobBoard**, an online recruitment platform connecting candidates and companies. Built three years ago by a small team, the application is based on the Symfony 6.4 framework and follows the MVC pattern: *one application, one database, one deployment*. Today, JobBoard can only be accessed from a Web browser.

The platform is a success: the number of users has grown sharply, new business clients have joined, and management now wants to offer JobBoard on other devices.

Link with the course — Chapter 1 learning outcomes mobilised

- LO 2** Distinguish Monolith, N-Tier, SOA and Microservices.
- LO 3** Identify the limits of a monolithic architecture.
- LO 4** Understand the principles of distributed systems.
- LO 5** Explain the role of Web services.

2 Scenario

Monday morning, steering meeting at TalentConnect.

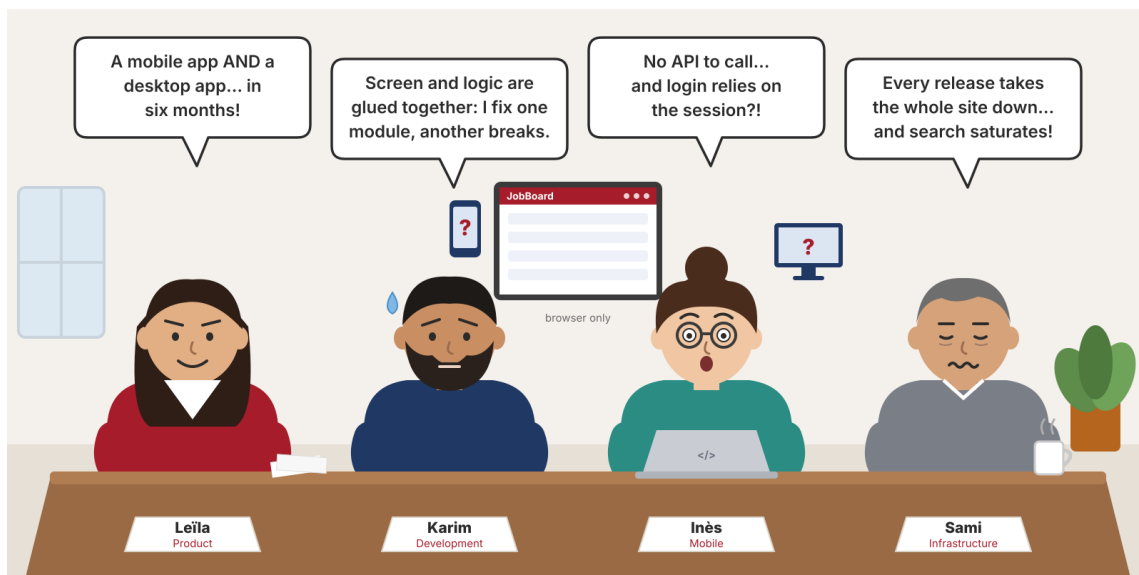


Figure 1 — Steering meeting at TalentConnect.

Leïla Head of Product

“The figures are clear: most of our candidates visit us from their smartphones and complain about having to go through the browser. The board wants a mobile app for candidates within six months. And our biggest clients, the recruitment agencies, are asking for a desktop application to process hundreds of applications a day.”

Karim the project’s original developer

“On paper, it’s simple: we keep the same logic and just change the screen. But in JobBoard, the screen and the logic are glued together. And whenever I touch the applications module, something breaks somewhere else: last week, a fix to the e-mail sending blocked the publication of job offers.”

Inès newly hired mobile developer

“I’ve gone through the code. I couldn’t find a single endpoint my mobile app could call: everything is designed to display pages in a browser. And user login relies on the browser session. How could an Android or desktop app use that?”

Sami Infrastructure Manager

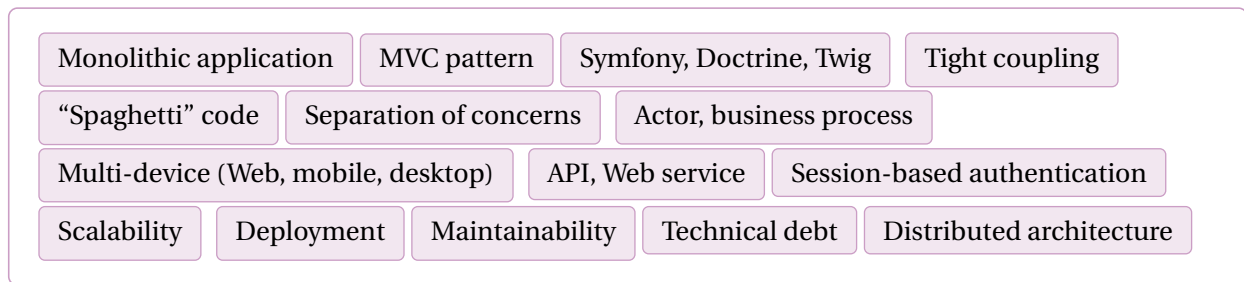
“Not to mention releases: every deployment means shutting down the whole platform. And during recruitment campaigns, job search saturates the server: the entire site slows down, including the recruiters’ area.”

Leïla Head of Product

“I don’t want a blind, complete rewrite. Before any decision, I want to understand what JobBoard actually does, why the application resists change, and which direction to take.”

The technical management entrusts your team with an **audit**. Starting from the JobBoard source code, *without running it*, you must understand the application, diagnose its limits in the face of new devices, and propose a well-argued architectural direction.

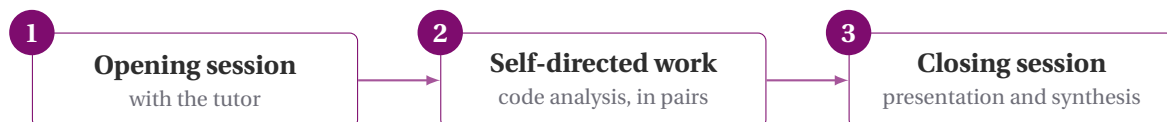
3 Keywords



4 Resources

- ▶ **JobBoard application source code (Symfony 6.4)**, provided: https://github.com/badi3a/AWD-Training/tree/Chapter_01/JobBoard_Symfony_6_4.
The analysis is *static*: you are not required to install or run the application.
- ▶ Course, Chapter 1: *Introduction to Distributed Architectures*.
- ▶ Official Symfony 6.4 documentation: <https://symfony.com/doc/6.4/index.html>
- ▶ Chapter 1, “Introduction to Distributed Application”, https://github.com/badi3a/AWD-Training/blob/Chapter_01/Chapter_1_Distributed_Architectures.pdf,

5 Required work (in pairs)



1. Opening session

As a group and with your tutor, starting from the context and the scenario, build:

1. the definitions of the keywords;
2. the problem statement of the prosit;
3. the constraints and expectations expressed by the actors of the scenario;
4. the solution hypotheses, to be confirmed or rejected during the closing session;
5. an action plan detailing the steps of your investigation and how the work is shared within the pair.

2. Self-directed work

Carry out your action plan. Your analysis of the source code must allow you to confirm or reject your hypotheses and to answer the problem statement. Support every finding with precise evidence from the code: *file, class, method, commented excerpt*.

3. Closing session

Each pair presents its approach, findings and proposal in **10 minutes**; the group discussion then compares the analyses and builds a synthesis with the tutor.

6 Deliverables

An **analysis report** (10 pages maximum) and a **presentation**. The report must contain at least:

- ▶ the problem statement, the hypotheses and their status (confirmed or rejected);
- ▶ a functional map of JobBoard: actors, main business processes and features;
- ▶ a representation of the application's current architecture;
- ▶ the coupling issues found in the code, illustrated with commented excerpts;
- ▶ an analysis of the limits of the current architecture: maintenance, scalability, deployment and openness to new devices;
- ▶ a target architecture proposal, with a diagram, justified by the needs expressed in the scenario.

Important. You are not asked to implement the solution. The quality of the analysis and of the reasoning matters more than quantity.

7 Assessment criteria

Criterion	What is assessed
PBL approach	Relevance of the problem statement, hypotheses and action plan.
Code analysis	Rigour of the analysis: precise and verifiable findings.
Architectural proposal	Justification of the proposal with respect to the needs.
Communication	Clarity of the report, the diagrams and the oral presentation.